

Sistema de Gestión de un Gimnasio

De Monolito a Microservicios con DDD

Daniel Alejandro Castro Escobar

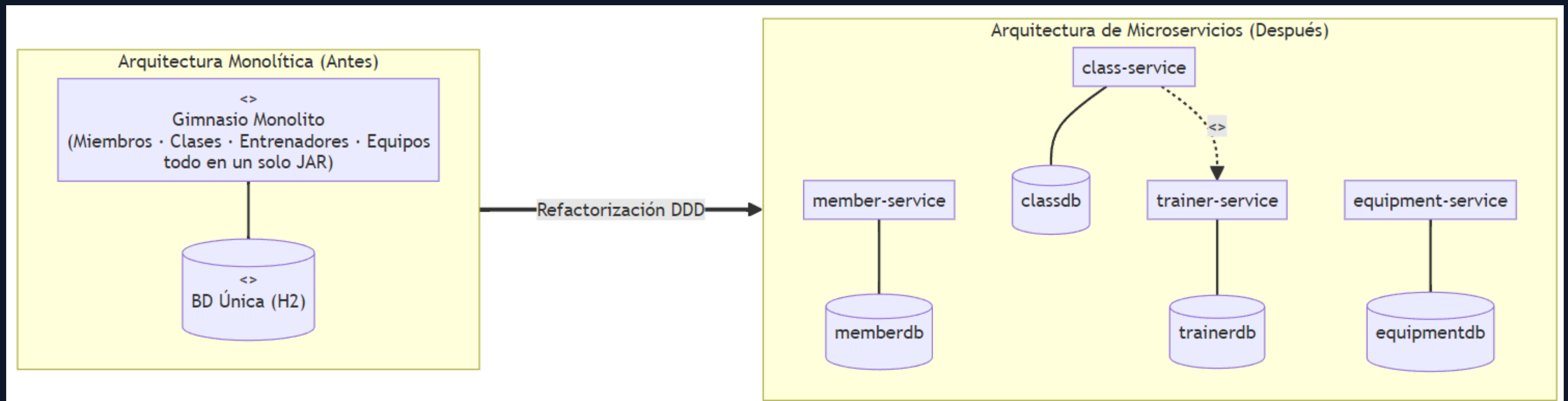
Laura Valentina Revelo Villareal

Katerine Valens Orejuela

Jeanpaul Atkinson Vidales Gallego

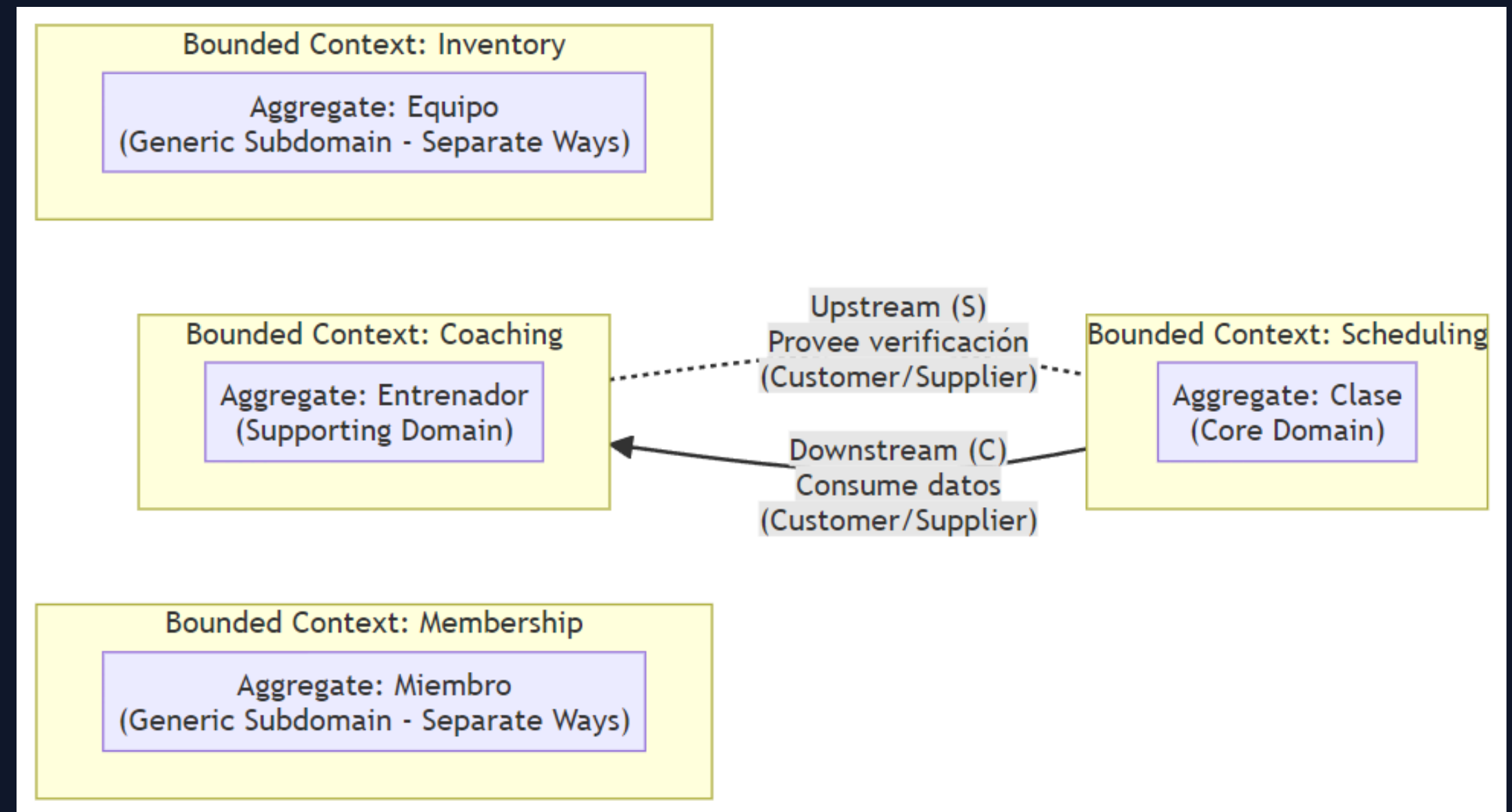
El Problema: El Monolito

- **Base de Datos Única:** 4 entidades centralizadas (Miembro, Clase, Entrenador, Equipo).
- **Lógica Acoplada:** Un único GimnasioService con toda la lógica de negocio mezclada.
- **API Centralizada:** Un único GimnasioController exponiendo todo bajo la misma ruta.
- **Dificultad de Escalamiento:** Imposible escalar dominios individuales por separado.



Análisis del Dominio

- **Dependencia Real:** La relación Clase-Entrenador (@ManyToOne) es la única dependencia entre entidades.
- **Autonomía:** Miembro y Equipo operan de forma aislada, sin acoplamiento de negocio.
- **Identificación de Contextos:** El análisis revela 4 Bounded Contexts naturales.
- **Decisión Arquitectónica:** Dividir el sistema en 4 microservicios integrando únicamente las áreas que lo exigen.



Los 4 Bounded Contexts

Microservicio	Bounded Context	Responsabilidad	Puerto
member-service	Membership	Alta y consulta de miembros	8081
class-service	Scheduling	Programar clases, validación remota	8082
trainer-service	Coaching	Alta y consulta de entrenadores	8083
equipment-service	Inventory	Alta y consulta de equipos	8084

Diagrama de Componentes UML

- **Interfaces Lollipops:** Cada microservicio expone sus propias APIs REST dedicadas.
- **Componentes Independientes:** El monolito se divide en artefactos Spring Boot separados.
- **Socket de Dependencia:** class-service declara una interfaz requerida hacia trainer-service.
- **Interacción:** Consumo síncrono para satisfacer reglas de negocio sin compartir código.

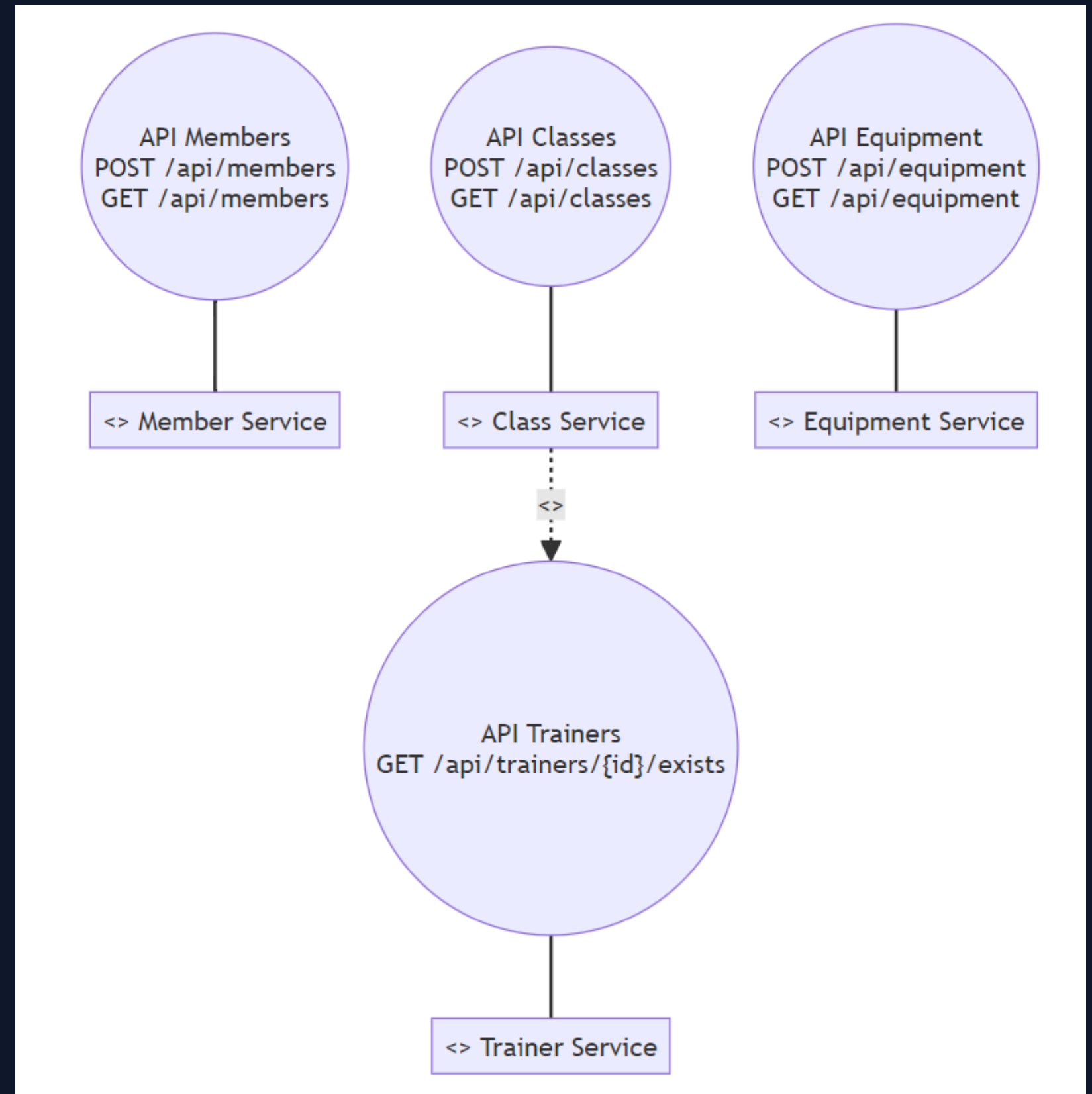
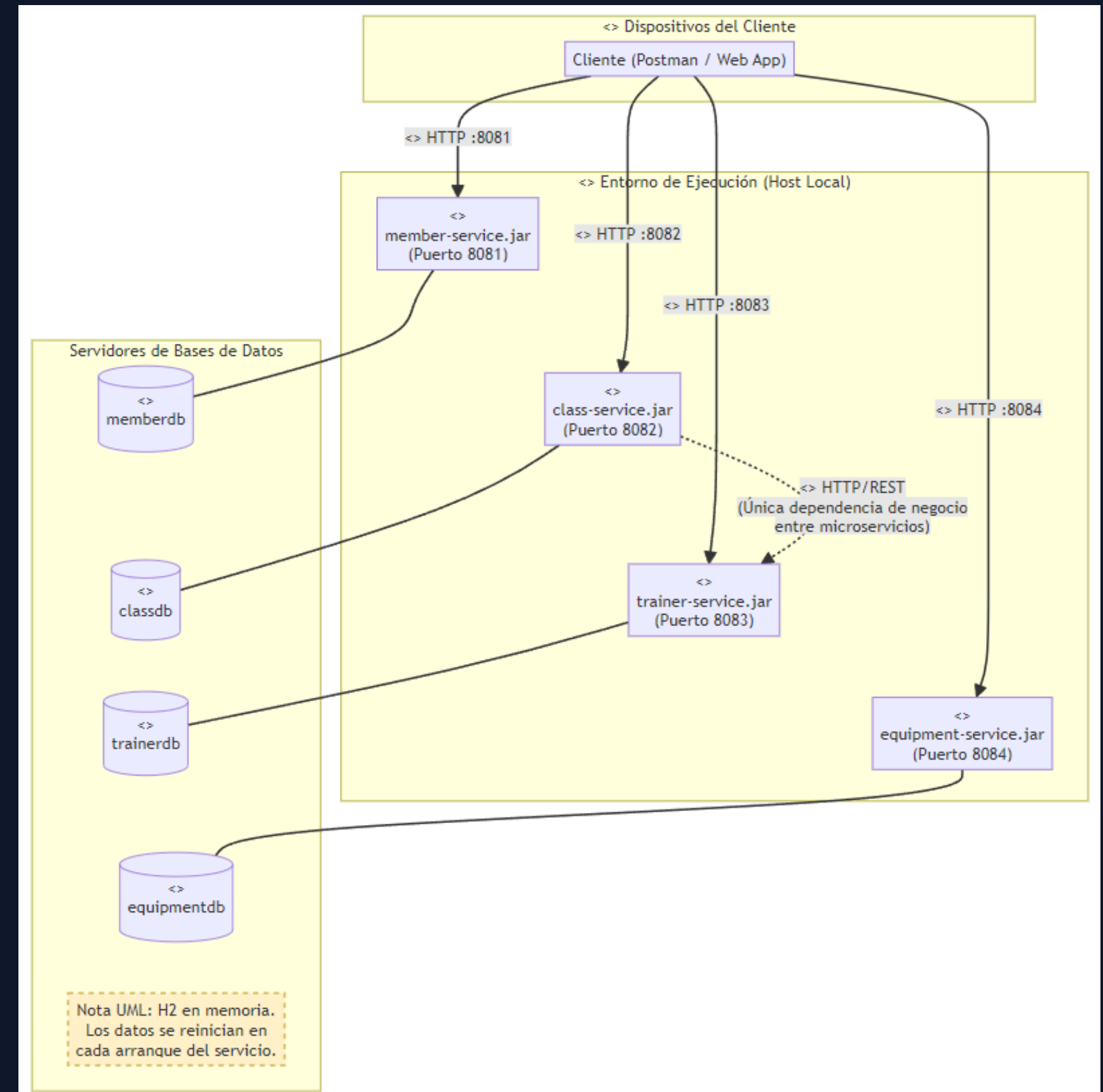


Diagrama de Despliegue UML

- **Procesos Aislados:** Cada microservicio ejecuta en su propio proceso y puerto lógico.
- **Persistencia Desacoplada:** 4 instancias independientes de H2 Database.
- **Sin Joins:** Imposibilidad física de realizar consultas cruzadas a nivel de base de datos.
- **Red como Canal:** Toda la comunicación inter-dominio viaja exclusivamente por HTTP.



Por qué NO se comparten entidades JPA



Referencia por ID

class-service no posee la entidad Entrenador. Solo almacena el entrenadorId para mantener la autonomía de los datos.



Patrón Hexagonal

El dominio define un VerificationPort. La infraestructura lo implementa con WebClient sin contaminar la lógica central.



Estructura Limpia

Capas de Dominio, Aplicación e Infraestructura repetidas rigurosamente en los 4 servicios para estandarizar el diseño.

Comunicación REST Síncrona

- **Validación (Write):** GET /api/trainers/{id}/exists para proteger la integridad al crear.
- **Enriquecimiento (Read):** GET /api/trainers/{id} para adjuntar detalles al listar clases.
- **Manejo de Errores:** Retorno de 409 Conflict si el recurso remoto no existe.
- **Degradación Elegante:** Si trainer-service cae, la lectura de clases omite el objeto entrenador sin fallar catastróficamente.

The image shows two screenshots of a REST client interface, likely Postman, demonstrating API interactions.

Top Screenshot: Successful POST Request

- Method:** POST
- URL:** http://localhost:8082/api/classes
- Body (raw JSON):**

```
{
  "nombre": "Clase Fantasma",
  "horario": "2026-08-10T19:00:00",
  "capacidadMaxima": 5,
  "entrenadorId": 1
}
```
- Response:** 201 Created, 136 ms, 345 B
- Body (raw JSON):**

```
{
  "id": 3,
  "nombre": "Clase Fantasma",
  "horario": "2026-08-10T19:00:00",
  "capacidadMaxima": 5,
  "entrenadorId": 1,
  "entrenador": {
    "id": 1,
    "nombre": "Carlos Rodríguez",
    "especialidad": "Yoga"
  }
}
```

Bottom Screenshot: 409 Conflict Error

- Method:** POST
- URL:** http://localhost:8082/api/classes
- Body (raw JSON):**

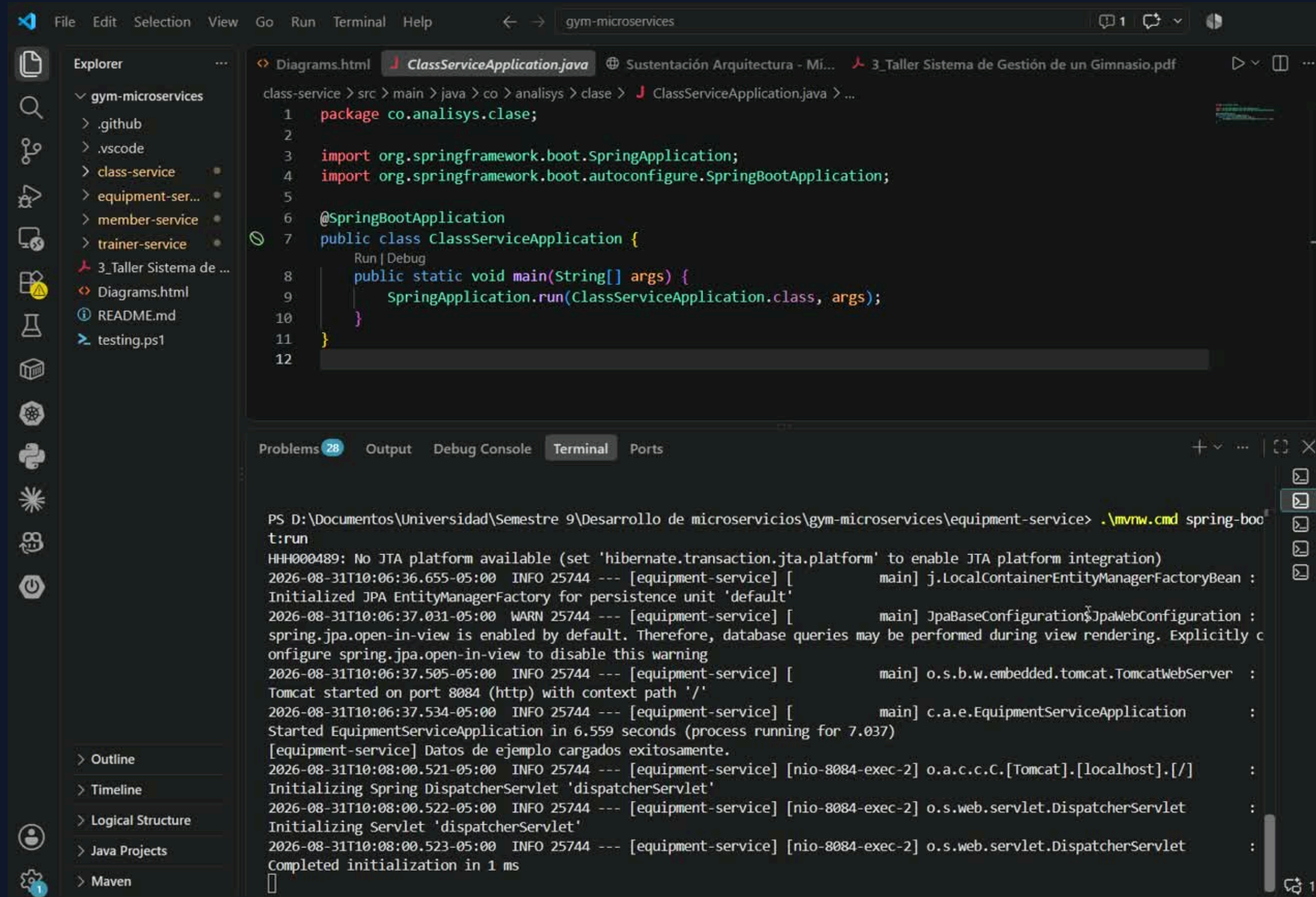
```
{
  "nombre": "Clase Fantasma",
  "horario": "2026-08-10T19:00:00",
  "capacidadMaxima": 5,
  "entrenadorId": 999
}
```
- Response:** 409 Conflict, 26 ms, 324 B
- Body (raw JSON):**

```
{
  "error": "Conflict",
  "message": "No se puede programar la clase: no existe un entrenador con id 999",
  "timestamp": "2026-08-31T14:55:07.4323353",
  "status": 409
}
```

Maapeo de Funcionalidades

Funcionalidad	Endpoint Monolito Original	Nuevo Endpoint Microservicio
Gestión de Miembros	/api/gimnasio/miembros	:8081/api/members
Gestión de Clases	/api/gimnasio/clases	:8082/api/classes
Gestión de Entrenadores	/api/gimnasio/entrenadores	:8083/api/trainers
Gestión de Equipos	/api/gimnasio/equipos	:8084/api/equipment

Demostración



The image shows a Visual Studio Code editor window with a dark theme. The Explorer sidebar on the left shows a project named 'gym-microservices' with subfolders like '.github', '.vscode', and 'class-service'. The main editor area displays the file 'ClassServiceApplication.java' with the following code:

```
1 package co.analisis.clase;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class ClassServiceApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(ClassServiceApplication.class, args);
10    }
11 }
12
```

Below the code editor, the Terminal panel is active, showing the output of a command executed in the 'equipment-service' directory. The command was `./mvnw.cmd spring-boot:run`. The output shows various log messages, including a warning about JPA configuration and the successful start of the application on port 8084.

```
PS D:\Documentos\Universidad\Semestre 9\Desarrollo de microservicios\gym-microservices\equipment-service> ./mvnw.cmd spring-boot:run
HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-08-31T10:06:36.655-05:00 INFO 25744 --- [equipment-service] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-08-31T10:06:37.031-05:00 WARN 25744 --- [equipment-service] [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-08-31T10:06:37.505-05:00 INFO 25744 --- [equipment-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8084 (http) with context path '/'
2026-08-31T10:06:37.534-05:00 INFO 25744 --- [equipment-service] [main] c.a.e.EquipmentServiceApplication : Started EquipmentServiceApplication in 6.559 seconds (process running for 7.037)
[equipment-service] Datos de ejemplo cargados exitosamente.
2026-08-31T10:08:00.521-05:00 INFO 25744 --- [equipment-service] [nio-8084-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-08-31T10:08:00.522-05:00 INFO 25744 --- [equipment-service] [nio-8084-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-08-31T10:08:00.523-05:00 INFO 25744 --- [equipment-service] [nio-8084-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
```

Decisiones de Diseño



4 Microservicios Estrictos: Alineados 1:1 con cada Bounded Context sin sub-dividir excesivamente la arquitectura.



Cero Entidades Compartidas: Ninguna clase se reutiliza entre repositorios, asegurando fronteras físicas y lógicas.



Acoplamiento Mínimo Viable: Interacción REST aplicada única y exclusivamente donde el modelo de negocio lo requiere.



Tolerancia a Fallos: Implementación de degradación elegante en consultas para no detener todo el sistema ante fallas parciales.

Gracias